

基于瑞萨的电能监测管理系统

摘要： 本系统基于 RA8D1_AI_KIT 开发板构建，实现了高精度电力参数监测与管理功能。硬件上，通过电流电压互感器隔离高压信号，经变压器降压和电平抬升电路适配至 RA8D1 内置 ADC 的 0-3.3V 输入范围，结合 RC 低通滤波与软件滑动平均算法抑制干扰，支持 0-250V 交流电压、0-6A 交流电流测量，误差 $\leq 1\%$ ，可分析 1-15 次谐波分量，总谐波失真度计算误差 $\leq 2\%$ ，还能实时计算有功功率、无功功率等参数及系统功耗、暂升暂降次数。

通信方面，借助 ESP8266 模块实现 2.4GHz 无线传输与自组网，采用 ESP8266wifi 模块将数据上传至基于 Qt 框架开发的上位机，具备断线自动重连功能。可视化交互上，通过 LCD 显示屏动态展示电压电流有效值、波形等信息，支持历史数据存储与查询。

系统创新体现在硬件架构（减少外部芯片与复杂电路，提升稳定性）、算法优化（优化 FFT 算法，构建电器“用电指纹”数据库）和功能集成（融合多项功能）。设计流程涵盖需求规划、硬件搭建等环节。

目前，系统完成了硬件实物与软件界面设计，可扩展接入温湿度传感器等，优化算法与通信协议。研发过程凸显了软硬件协同的重要性，为电力监测领域提供了高效解决方案。

目录

1 作品概述.....	4
1.1 功能与特性	4
1.2 应用领域	5
1.3 主要技术特点	5
1.4 主要性能指标	6
1.5 主要创新点	7
1.6 设计流程	7
2 系统组成及功能说明.....	9
2.1 系统整体介绍	9
2.2 硬件系统介绍	10
2.2.1 硬件整体介绍.....	10
2.2.2 电路各模块介绍.....	10
2.3 软件系统介绍	12
2.3.1 软件整体介绍.....	12
2.3.2 软件各模块介绍.....	13
3 云平台与上位机介绍.....	20
3.1 数据传输整体介绍	20
3.1.1 下位机向 ESP8266 的数据打包与传输.....	20
3.2 云平台上的部署	22
3.3 QT 上位机部署.....	22
3.4 云平台介绍	23
3.4.1 OneNET 云平台的优势	23
3.4.2 OneNET 中数据传输的路径	23
3.5 QT 上位机介绍.....	24

3.5.1 QT 与 OneNET 建立联系	24
3.5.2 数据清洗.....	24
3.5.3 数据解析.....	25
4 完成情况及性能参数.....	27
4.1 整体介绍	27
4.2 工程成果（分硬件实物、软件界面等设计结果）	29
4.2.1 电路成果.....	29
4.2.2 软件成果.....	31
4.3 特性成果	33
5 总结.....	36
5.1 可扩展之处	36
5.2 心得体会	36
参考文献.....	38

1 作品概述

1.1 功能与特性

基于 RA8D1_AI_KIT 开发板的电力参数监测系统完整实现了高精度电参数测量、可靠信号调理、稳定物联网通信及友好可视化交互功能。系统通过电流电压互感器隔离高压信号，将高压信号通过变压器降压到可接受范围并采用电平抬升电路将信号适配至 RA8D1 内置 ADC 的 0-3.3V 输入范围，结合硬件 RC 低通滤波与软件滑动平均算法有效抑制工频噪声与高频干扰。在电参数测量方面，系统支持 0-250V 交流电压与 0-6A 交流电流测量，测量误差 $\leq 1\%$ ，能精确分析 1-15 次谐波分量，总谐波失真度（THD）计算误差 $\leq 2\%$ ，同时实时计算有功功率（P）、无功功率（Q）、视在功率（S）及功率因数（PF），同时通过以上数据计算系统功耗以及暂升暂降次数等数值。

物联网通信功能通过 ESP8266 模块实现，2.4GHz 通信与自组网能力，采用无线传输形式将数据上传至基于 Qt 框架开发的上位机中，并具备断线自动重连机制确保数据完整性。可视化交互方面，系统基于 LCD 显示屏构建可视化界面，动态展示电压电流有效值、电压电流波形、功率参数及谐波频谱，并提供历史数据存储与查询功能，支持按时间范围筛选历史记录。该方案充分利用

RA8D1_AI_KIT 开发板的高性能计算能力与丰富接口资源，在单一平台上实现了从数据采集、处理到传输、展示的全流程功能集成。

1.2 应用领域

在当下双碳目标的引领下，要求新建项目需满足“可观、可测、可调、可控”，以此提升其接入电网的承载力与调控能力。在此背景下，基于 FFT 算法的单相电能分析仪系统发挥着关键作用。在太阳能光伏系统中，该系统能够实时且精准地监测光伏板输出的电力参数。一方面，确保光伏逆变器输出的交流电符合电网接入标准，保障发电系统能高效、稳定地并入电网，避免因谐波问题导致的电网污染及设备故障，提升光伏系统在电网中的兼容性与稳定性。另一方面，能敏锐捕捉到光伏板因老化、局部遮挡等异常状况引发的电力参数波动，及时预警维护，保护发电设备安全，延长其使用寿命。此外，在校园宿舍用电管理领域，能够高精度采集宿舍总进线或各回路的电压、电流有效值及功率参数。当学生使用大功率电器时，系统会瞬间检测到电流突增、功率超过阈值的情况并立即发出告警并切断供电，同时 ESP8266 模块迅速将告警信息同步至宿舍管理终端。有效避免因超负荷用电引发的电路过载，从根源上杜绝此类危险事件的发生。

1.3 主要技术特点

高性能硬件平台：采用逐飞科技 RA8D1_AI_KIT 开发板，搭载主频 240MHz 的 RA8D1 高性能 MCU，集成 32MB SDRAM 与 8MB Flash，内置 DSP 与 FPU 运算单元，结合硬件加速特性，FFT 算法处理效率较传统方案提升 30%，确保电参数实时分析与复杂运算高效执行。

精准信号处理架构：构建电流 / 电压互感隔离与电平抬升电路，将 0-250V 交流电压、0-6A 交流电流信号精准调理至 ADC 适配范围；配合硬件 RC 低通滤波与软件滑动平均算法，有效抑制 50Hz 工频干扰与高频噪声。

智能频谱分析算法：基于优化后的 FFT 算法，可实时解析 1-15 次谐波分量，同步计算有功功率、无功功率、视在功率、及功率因数等 12 项电力参数；通过动态窗函数调整，适应不同负荷场景下的高精度分析需求。

低功耗通信网络：采用 ESP8266 模块，具有 2.4GHz 通信与自组网能力，结合 RA8D1 深度睡眠模式，实现 7×24 小时在线监测的低功耗运行；

可视化上位机系统：基于 Qt 框架开发的上位机系统，动态呈现电压 / 电流波形、频谱图谱及功率参数；集成历史数据存储与检索功能，支持按时间粒度（分钟 / 小时 / 日）查询与导出，满足能源管理与故障追溯需求。

双平台数据上传：采取双云平台制降低系统崩溃风险，主平台使用 OneNET 云平台，采取 MQTT 协议与 JSON 数据格式进行上传，在规则引擎中采取 MQ 消息队列的方式存储数据，并且采取 8883 加密端口与 TLS1.2 证书验证，保障数据传输。副平台采取自建云平台，如果主平台发送崩溃可转入副平台维持数据上传与下发。

1.4 主要性能指标

表 1.1 主要性能测试结果与误差分析

参数类型	0.701 (A)	1.21 (A)	1.841 (A)	2.041 (A)	3.43 (A)	3.598 (A)
电流实测 (A)	0.689	1.13	1.836	2.023	3.48	3.57
绝对误差 (%)	1.711	1.652	0.271	0.881	1.457	0.778
电压 (V)	223.1	222.1	222.2	222.5	220.5	220.3
电压实测 (V)	223.3	223.1	222.6	222.2	220.3	220.2
绝对误差 (%)	0.089	0.45	0.18	0.134	0.09	0.045
有功功率 (W)	122.5	206.8	318.5	335.3	709	738.2
有功功率实测 (W)	121.1	204.5	316.5	338.8	702.6	745.2
绝对误差 (%)	1.142	1.112	0.627	1.043	0.902	0.948
功率因数	0.787	0.772	0.779	0.782	0.941	0.935

1.5 主要创新点

(1) 硬件架构创新:

借助 RA8D1_AI_KIT 开发板核心板,减少外部芯片与复杂电路,降本减容,提升稳定性与可靠性。通过电流 / 电压互感、电平抬升电路,适配不同电力信号范围,灵活调整硬件参数,应对多样监测需求。通过 ESP8266 连接上位机,满足云端监控和组网的需求

(2) 算法优化创新:

FFT 高效优化:深度优化 FFT 算法,处理效率提升,可快速对大量电力信号频谱分析,精准获取电压、电流有效值及谐波分量,支撑电力参数实时监测。

谐波诊断应用:精准分析 1 - 15 次谐波分量,构建电器“用电指纹”数据库。

(3) 功能集成创新:

多功能融合:深度融合电参数精确测量、信号调理抗干扰、物联网通信、可视化交互功能。

1.6 设计流程

(1) 需求规划:明确 0-250V 电压、0-6A 电流高精度测量,1-15 次谐波分析等核心需求,结合开发板特性规划整体方案。

(2) 硬件搭建:设计高压变压与抬升电路适配 ADC 输入,配置开发板接口连接 ESP8266 模块、LCD 屏等外设。

(3) 软件开发:配置 ADC 采集信号,优化 FFT 算法处理数据,编写限电、识别等功能逻辑。

(4) 通信交互:实现本地 LCD 显示、通过串口与 ESP8266 通信,利用 ESP8266 将打包好的数据发布到 OneNET,QT 上位机订阅获取发布的数据包并进行数据解析,最后在 QT 上位机上进行显示。

(5) 调试优化:测试硬件精度与通信稳定性,校准算法误差,模拟场景优化功能。

(6) 集成部署：整合软硬件及云端，优化功耗后部署至实际场景运行。

2 系统组成及功能说明

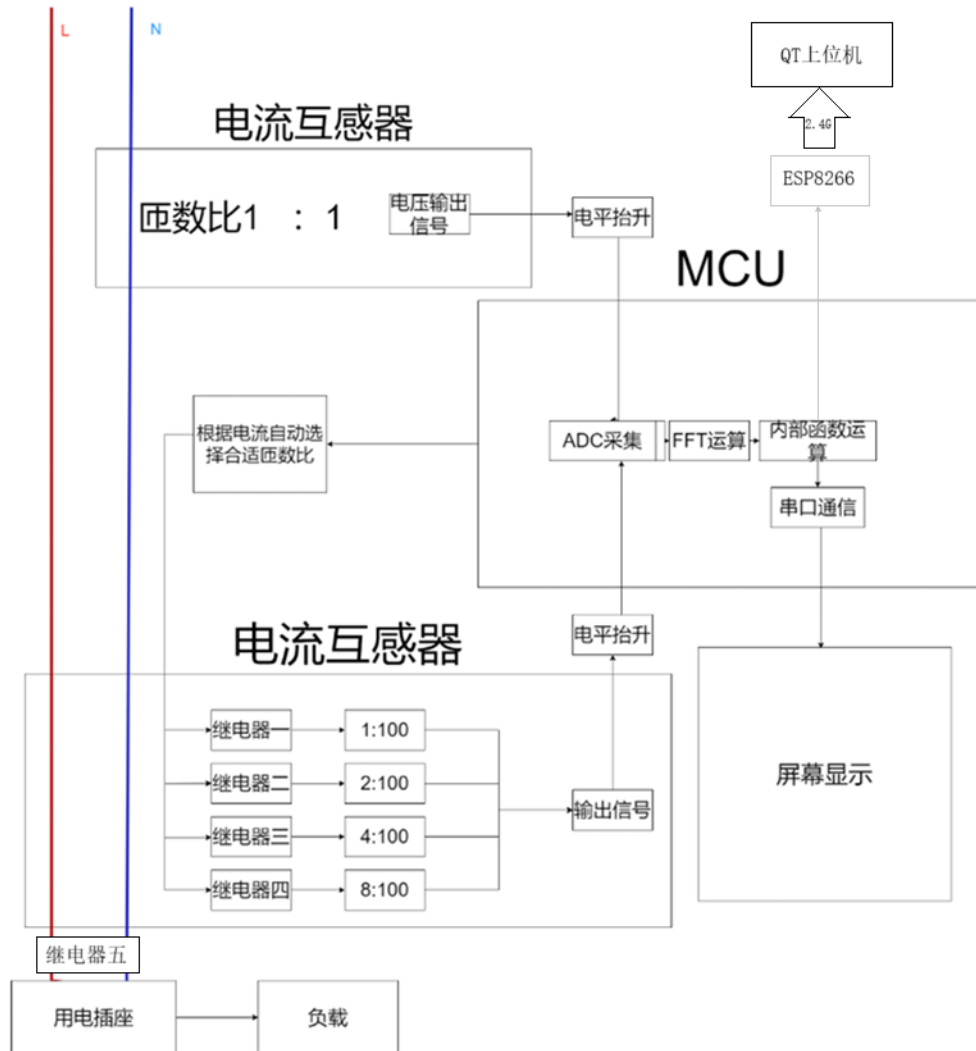


图 2.1 系统总体组成示意图

2.1 系统整体介绍

系统采用电流型电压互感器，在负载工作时，电压通过一定匝数比的电压互感器得到固定电压信号。这个固定电压信号通过偏置电路进行抬升，使其达到单片机内部 AD 转换器可测量的电压范围。同样地，当负载工作时，电流通过一定匝数比的电流互感器后，由电阻转化为电压信号。这个电压信号也通过偏置电路进行抬升，使其达到单片机内部 AD 转换器可采集的电压范围。

信号通过 ADC 进入单片机后，分别进行电流有效值计算，数字有功功率、功率因数测量，谐波系数测量，暂升暂降监控等处理。处理完成的数据除了在

LCD 屏幕上显示出来以外，还会同时通过 ESP8266 上传到 QT 上位机，以便进行远程监控与组网。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

主控与存储：瑞萨 RA8D1 MCU 提供算力支撑，保障 FFT 算法及 10kHz 采样率下的信号处理效率；32MB SDRAM 缓存 FFT 运算数据，8MB Flash 存储程序、历史数据及电器“用电指纹”，支持断电留存。

接口与辅助：含 SWD 调试、TYPE-C USB、DVP 接口；复位 / BOOT 按键便于调试，板载 LED 及电源指示灯反馈系统状态。扩展板模块

电源与采集：支持 7-26V 宽压输入，双端子输出稳定电源；通过 ADC 引脚连接调理电路，实现 0-250V 电压、0-4A 电流采集，板载 RC 滤波抑制高频干扰。电流 / 电压互感器隔离高压信号，经电平抬升适配 ADC 0-3.3V 输入，多级 RC 滤波抑制工频及谐波干扰；扩展板 IO 口连接继电器，检测到违规用电时自动断电，实现智能限电。

通信与扩展：UART 接 ESP8266 模块，SPI 驱动 LCD 本地显示，预留 WiFi 接口；含编码器、电机控制接口及麦克风、扬声器、六轴传感器，支持功能拓展。外围适配电路

2.2.2 电路各模块介绍

1) 电压互感器

所用的电压互感器允许通过最大电流为 2mA。因为市电为 220V，需要串联一个电阻将电压信号转换为 2mA 一下的电流，电阻选用 110KΩ，最终得到的电流为： $220/110000=0.002A$ ，在电压互感器另一侧并联 100Ω 的电阻将电流值转化为电压值，得到的电压值为 0.20V。

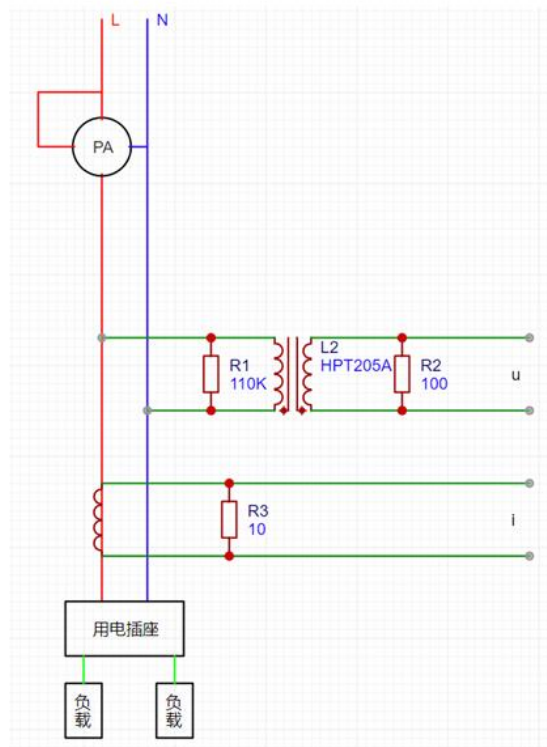


图 2.2 电压电流互感装置原理图

2) 电流互感器

将匝数比设定为 1001，4A 的电流转换为 40mA，再通过负载电阻的方式将电流转化为电压。要使用单片机内部 AD 采集电压，故负载电阻选取 10Ω ， $U=I/R$ 转化电压为 400mV。

3) 电平抬升电路

运算放大器采用 OP07，具有极低的输入失调电压，这有助于确保在高精度应用中的准确性和稳定性。根据资料，其输入失调电压可低至 10uV。OP07 的失调电压温漂极低，仅为 $0.2\mu\text{V}/^\circ\text{C}$ ，这意味着在不同温度下，其性能变化非常小，有利于保持电路的长期稳定性。并且还拥有较宽的电压工作范围，从 $\pm 3\text{V}$ 到 $\pm 22\text{V}$ ，这使得它在不同的电源环境下都能保持稳定的性能。

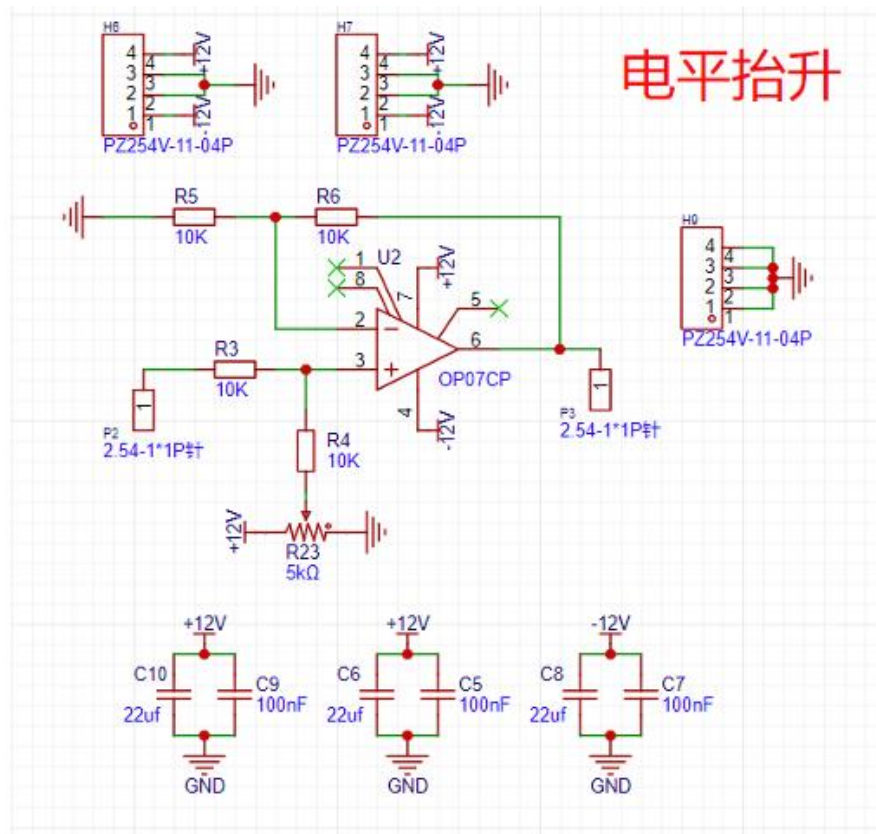


图 2.3 电平抬升原理图

2.3 软件系统介绍

2.3.1 软件整体介绍

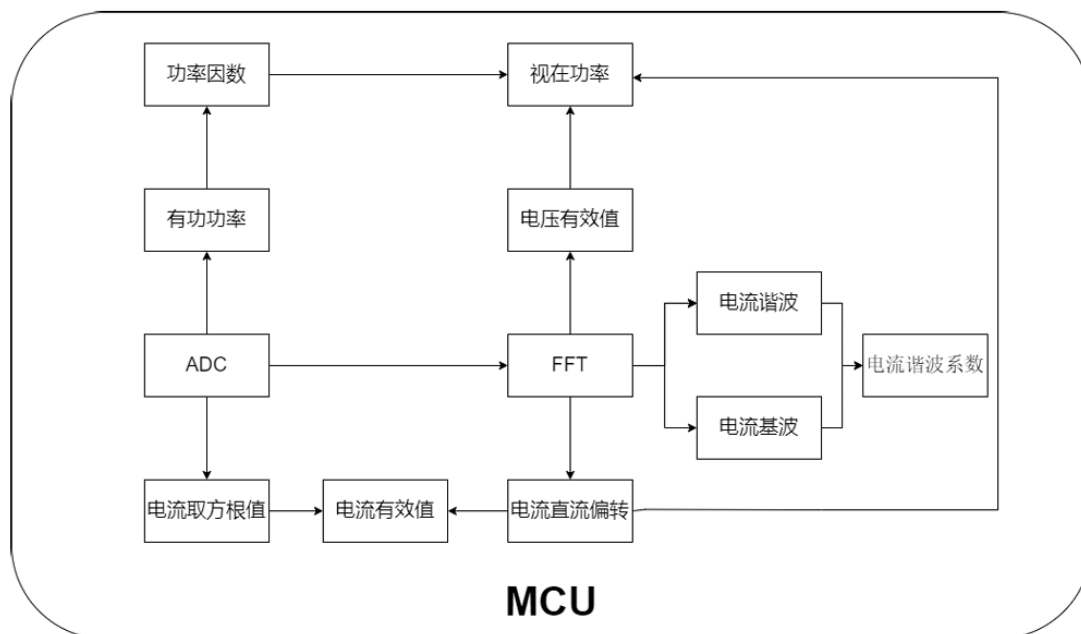


图 2.4 程序流程图

数据采集与处理：配置 RA8D1 的 ADC 模块以 5120Hz 同步采集电压 / 电流信号，通过定时器触发与 DMA 双缓冲区 “乒乓操作” 避免数据覆盖；基于 RA8D1 的 DSP 与 FPU 单元，调用优化后的 FFT 算法（1024 点运算）实现频谱分析，提取 1-15 次谐波分量，进而计算电压 / 电流有效值、有功功率、功率因数及总谐波失真度（THD），其中有效值通过基波与谐波分量平方和开方求得，THD 聚焦 12 次以内谐波计算。

通信协议实现：通过 UART 接口驱动 ESP8266 模块（SX126X），采用 Wifi 传输方式入网确保持续接收；采集数据封装为 JSON 格式（含电压、电流、功率等字段），传输至 QT 上位机进行统一监测，支持断线自动重连。

2.3.2 软件各模块介绍

1) ADC 模块

ADC_Init() 配置 RA8D1 的 ADC 为双通道同步采样（通道 1 采集电压信号，通道 2 采集电流信号），12 位分辨率，绑定定时器。使用定时器触发 ADC 实现 5120Hz 采样，确保电压与电流信号的严格同步，避免相位差引入误差。设置 DMA（直接内存访问）传输，将 ADC 采样数据存入双缓冲区（Buffer A 和 Buffer B），实现“乒乓操作”，避免数据覆盖。

电流信号配置为同步采样模式，采样率设为 5120Hz，分辨率为 12 位。使用定时器触发 ADC 采样，确保电压与电流信号的严格同步，避免在采样时引入相位差引入误差。

```
void ADC_Init(void)
{
    fsp_err_t err;
    err = R_ADC_Open(&g_adc0_ctrl, &g_adc0_cfg);

    err = R_ADC_ScanCfg(&g_adc0_ctrl, &g_adc0_channel_cfg);
    assert(FSP_SUCCESS == err);
}
```

```

double Read_ADC_Voltage_Value0(void)
{
    uint16_t adc_data0;
    double a0;
    (void)R_ADC_ScanStart(&g_adc0_ctrl);
    while (!scan_complete_flag) //等待转换完成标志
    {
        ;
    }
    scan_complete_flag = false; //重新清除标志位
    /* 读取通道 0 数据 */
    R_ADC_Read(&g_adc0_ctrl, ADC_CHANNEL_4, &adc_data0);
    /* ADC原始数据转换为电压值 (ADC参考电压为3.3V) */
    a0 = (double)(adc_data0*220.0f/1351.35f) - correction_vot;// 220/4095*3.3
    return a0;
}

void gpt0_callback(timer_callback_args_t* p_args)
{
    /*定时器溢出事件*/
    if(TIMER_EVENT_CYCLE_END== p_args->event)
    {
        /*---ADC采样与处理
        sp_adc_value0[adc_value_indictor] = Read_ADC_Voltage_Value0();
        sp_adc_value1[adc_value_indictor] = Read_ADC_Voltage_Value1();
        sp_y_front[adc_value_indictor] = (uint16_t)(sp_adc_value0[adc_value_indictor] * 0.2f+120); //电压图像生成辅助
        sp_y_behind[adc_value_indictor] = (uint16_t)(sp_adc_value1[adc_value_indictor] * 0.2f+105); //电流生成辅助
        if(++adc_value_indictor==FFT_LEN)
        {
            adc_value_indictor = 0;
            fft_volt_proc();
            fft_elec_proc();
        }
        */
    }
}

```

图 2.5 ADC 功能主要程序

2) FFT 频谱分析子模块（算法处理层）

FFT 的输入为离散时域信号 $x[n]$ ($n = 0, 1, \dots, N - 1$), 其频域表示为:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-\frac{j2\pi kn}{N}}, \quad k = 0, 1, \dots, N - 1 \quad (2-1)$$

其中: $X[k]$ 是频域中第 k 个频率分量的复数形式, N 为采样点数

FFT 的幅度计算

FFT 输出的每个 $X[k]$ 是复数, 其幅度 (模长) 表示对应频率分量的幅值:

$$\text{幅度}[k] = |X[k]| = \sqrt{\text{Re}(X[k])^2 + \text{Im}(X[k])^2} \quad (2-2)$$

物理意义: 幅度反映了信号中频率为 $f_k = k \cdot f_s / N$ (f_s 为采样率) 的正弦波强度。

幅度与信号能量的关系

单频信号: 若时域信号 $x[n] = A \cos(2\pi f n / f_s)$, 其 FFT 幅度在对应频率 f 处为 $A \cdot N/2$ (因正负频率对称)

幅度修正: 实际应用中需对 FFT 的结果进行归一化, 得到真实幅度:

$$\text{实际幅度} = \frac{2 \cdot |X[k]|}{N} \quad \left(k \neq 0 \text{ 且 } k \neq \frac{N}{2} \right) \quad (2-3)$$

例外：直流分量（ $k=0$ ）无需除以 2

FFT 用于幅度分析的步骤

信号采样：以采样率 f_s 采集 N 点信号 $x[n]$ 。

FFT 计算：得到 $X[k]$ 。

幅度提取：计算 $|X[k]|$ ，并修正为实际幅度 $(2 |X[k]| / N)$ 。

频率周映射：频率分辨率为 $\Delta f = f_s / N$ ， $f_k = k \cdot \Delta f$ 。

Proc 函数初始化 ARM CMSIS - DSP 库的 1024 点实数 FFT 实例，利用 RA8D1 的 FPU 加速运算，加载复数运算与幅值计算函数。从 ADC 缓冲区读取时域数据（虚部置零），调用 `arm_rfft_fast_f32()` 执行变换，提取 1 - 15 次谐波的幅值与相位，并按规则归一化（直流分量 / 1024，交流分量 / 512）。

```

void fft_volt_proc()
{
    //1. 将浮点数切换为复数类型
    for(int i=0; i<FFT_LEN; i++) {
        FFT_VOLT_INPUT[i*2] = (float) sp.adc_value0[i]; //实部
        FFT_VOLT_INPUT[i*2+1] = 0;
    }
    //2. 调用FFT进行计算
    arm_cfft_radix4_f32(&scfft, FFT_VOLT_INPUT);
    //3. 将复数数据转换为实数, 取模
    arm_cmplx_mag_f32(FFT_VOLT_INPUT, FFT_VOLT_OUTPUT, FFT_LEN);
    for(int k=0; k<FFT_LEN; k++) {
        freamp[k] = FFT_VOLT_OUTPUT[k] * fft_correction_vot / 1000.0f;
    }
}

void fft_elec_proc()
{
    //1. 将浮点数切换为复数类型
    for(int i=0; i<FFT_LEN; i++) {
        FFT_ELEC_INPUT[i*2] = (float) sp.adc_value1[i]; //实部
        FFT_ELEC_INPUT[i*2+1] = 0;
    }
    //2. 调用FFT进行计算
    arm_cfft_radix4_f32(&scfft, FFT_ELEC_INPUT);
    //3. 将复数数据转换为实数, 取模
    arm_cmplx_mag_f32(FFT_VOLT_INPUT, FFT_ELEC_OUTPUT, FFT_LEN);
}
  
```

图 2.6 FFT 功能主要程序

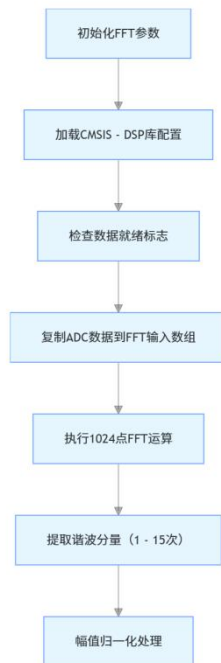


图 2.7 FFT 运算流程图

3) 电力参数计算子模块（应用逻辑层）

交流电压有效值^{错误!未找到引用源。} 计算：

设含有谐波的电压表达式为：

$$u(t) = U_0 + U_{1m} \sin(\omega t + \varphi_1) + U_{2m} \sin(2\omega t + \varphi_2) + U_{3m} \sin(3\omega t + \varphi_3) + \dots + U_{nm} \sin(n\omega t + \varphi_n)$$

其中 U_0 为直流分量， U_{nm} 为第 n 次谐波的幅值。其有效值 U 的计算公式为：

$$U = \sqrt{U_0^2 + \frac{U_m^2}{2} + \frac{U_{2m}^2}{2} + \dots + \frac{U_{nm}^2}{2}} \quad (2-4)$$

在工作时，电压互感器将电压信号转换后，通过板载 AD 的通道一进行采样。

具体步骤如下：

- 1、采样：对转换后的电压信号采样 1024 个点，采样率为 5120Hz。这样可以确保信号的实际分辨率为 5Hz。
- 2、FFT 运算：对采集到的 1024 点电压数组进行快速傅里叶变换（FFT）运算，得到电压信号的幅频特性曲线^{错误!未找到引用源。}。
- 3、提取谐波电压值：依次取出曲线中的一次、二次、三次……十二次谐波的电压值。

4、数据处理：将基波的平方与对这后面的二至十二次个电压值进行平方除以二然后相加。求出平方和的平方根。

(2) 交流电流有效值 错误!未找到引用源。 计算：

设含有谐波的电流表达式为

$$i(t) = I_0 + I_{1m} \sin(\omega t + \theta_1) + I_{2m} \sin(2\omega t + \theta_2) + I_{3m} \sin(3\omega t + \theta_3) + \dots + I_{nm} \sin(n\omega t + \theta_n)$$

其中 I_0 为直流分量， I_{nm} 为第 n 次谐波的幅值。其有效值 I 的计算公式为：

$$I = \sqrt{I_0^2 + \frac{I_m^2}{2} + \frac{I_{2m}^2}{2} + \dots + \frac{I_{nm}^2}{2}} \quad (2-5)$$

在负载工作时，电压互感器将电压信号转换后，通过板载 AD 的通道二进行采样。具体步骤如下：

- 1、采样：对转换后的电压信号进行 1024 点采样，采样率为 5120Hz。这样可以确保信号的实际分辨率为 5Hz。
- 2、FFT 运算：对采集到的 1024 点电压数组进行 1024 点快速傅里叶变换（FFT）运算，得到电压信号的幅频特性曲线。
- 3、提取谐波电流值：依次取出曲线中的一次、二次、三次……十二次谐波的电流值。
- 4、数据处理：将基波的平方与对这后面的二至十二次个电流值进行平方除以二然后相加，求出平方和的平方根。

(3) 数字有功功率、功率因数测量：

$$S = \frac{1}{T} \int_0^T ui \, dt \quad (2-6)$$

$$P = UI \quad (2-7)$$

$$\cos\theta = \frac{P}{S} \quad (2-8)$$

利用片上高速 AD 转换器对电压和电流值进行同步采样，得到电流和电压值的采集数组，对电压、电流值数组进行积分运算，计算出有功功率。具体步骤如下：

- 1.同步采样：利用片上高速 AD 转换器，对电压值和电流值进行同步采样，分别得到电压值数组和电流值数组。
- 2.积分运算：对采集来的电压值数组和电流值数组进行积分运算,求出有功功率,积分运算可以采用数值积分方法,如梯形积分法和辛普森积分法。

(4) 计算总谐波失真度（THD）

$$THD_x = \frac{\sqrt{U_{m2}^2 + U_{m3}^2 + \dots + U_{mx}^2}}{U_{o1}} \times 100\% \quad (2-9)$$

公式中的 U_{o1} 与 U_{mn} 均为有效值

1. 信号采集：片内 ADC 采集经过调理后的信号。
2. FFT 运算：利用快速傅里叶变换（FFT）将时域波形数据转换为频域电压数据。
3. 提取基波与谐波电压幅值：通过比较各频率点的电压大小，
4. 计算总谐波失真度（THD）：只处理至 12 次谐波。

```

void volt_rms_calcu()
{
    float volt_square_sum = 0;
    for(int i=0; i<FFT_LEN; i++) {
        volt_square_sum += pow(sp.adc_value0[i], 2.0);
    }
    float volt_final_sum = volt_square_sum / FFT_LEN;
    sp.volt_rms = sqrt(volt_final_sum);
}

//电流有效值计算
void elect_rms_calcu()
{
    float a, b; //根据变压器匝数 计算修正值
    if(sp.relay_mode == 0) {
        a = correction_elec * 0.55;
        b = correction_elec_down * 0.55;
    }
    else if(sp.relay_mode == 1) {
        a = correction_elec * 0.9;
        b = correction_elec_down * 0.9;
    }
    else if(sp.relay_mode == 2) {
        a = correction_elec * 1.8;
        b = correction_elec_down * 1.8;
    }
    else if(sp.relay_mode == 3) {
        a = correction_elec;
        b = correction_elec_down;
    }
    current_value_final_num = 0;

    for(int i=0; i<FFT_LEN; i++)
    {
        current_value_sqrt_num = (sp.adc_value1[i] - up_elec); //电压减去抬升电平
        current_value_final_num += current_value_sqrt_num*current_value_sqrt_num; //计算平方和
    }
    current_value_final_num = current_value_final_num/FFT_LEN; //平方和除以1020;
    sp.elect_rms = sqrt(current_value_final_num) * a - b; //计算得到的有效值
    if(sp.elect_rms>25.0f) sp.elect_rms=0.30f;
}
  
```

图 2.8 电压与电流有效值计算

```
// 获取各种功率参数
void get_power_data()
{
    // 视在功率
    float pow_aveg_sum = 0;
    for(int i = 0; i < FFT_LEN; i++){
        pow_aveg_sum += sp.elect_rms * sp.volt_rms;
    }
    sp.apparent_power = pow_aveg_sum / FFT_LEN;
    if(sp.system_mode == 2){
        // 角度转换: 将角度转换为弧度
        float phase_radians = sp.phase_data * (PI / 180.0f);

        // 有功功率
        sp.active_power = sp.apparent_power * cos(phase_radians);

        // 无功功率
        sp.reactive_power = sp.apparent_power * sin(phase_radians);

        // 功率因数
        sp.power_factor = sp.active_power / sp.apparent_power; // 计算功率因数
    }
}
```

图 2.9 功率计算

3 云平台与上位机介绍

3.1 数据传输整体介绍

RA8D1_AI_KIT 开发板将数据打包，通过 ESP8266 将打包后的数据发布到云平台，QT 上位机订阅云平台上的相应频道获取打包后的数据，之后 QT 上位机解析数据，并以图形化的方式表现出来

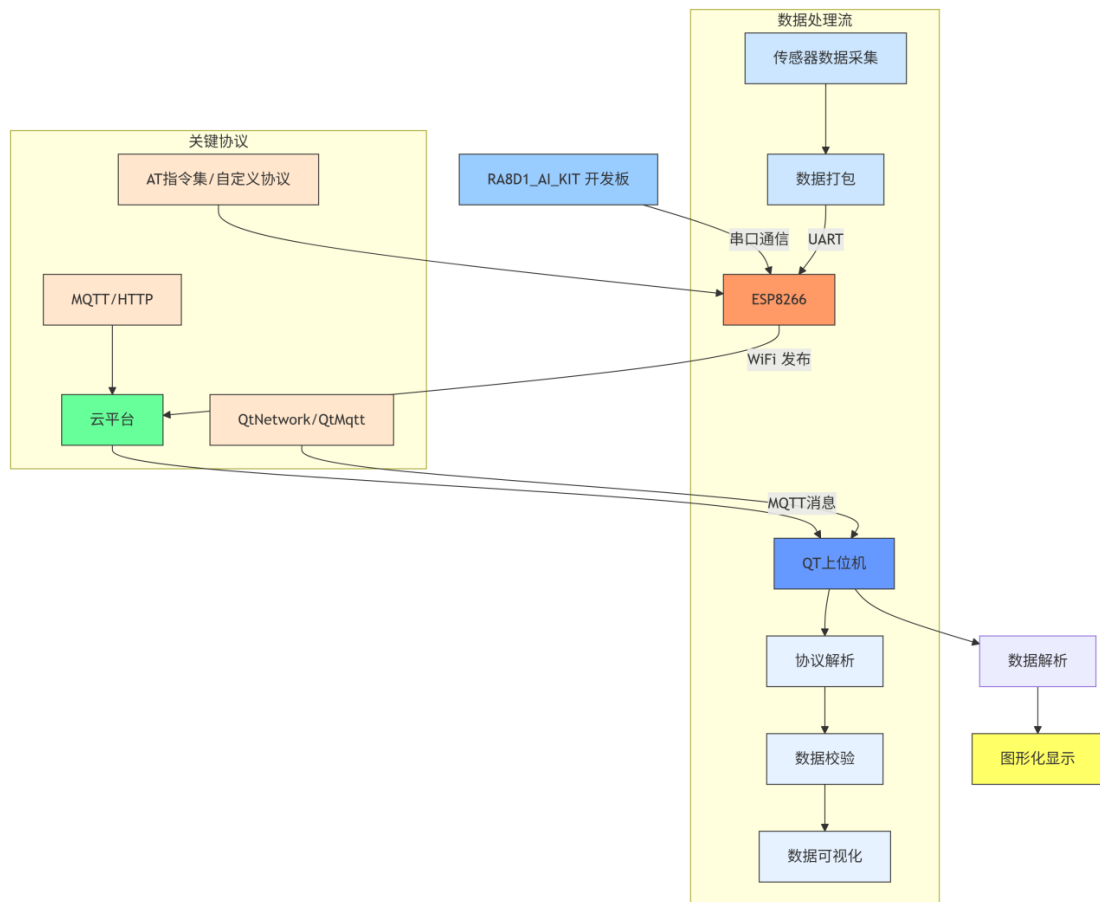


图 3.1 系统整体介绍

3.1.1 下位机向 ESP8266 的数据打包与传输

将数据打包成“A%06.2f?B%06.2f?C%05.2f?D%07.2f?E%09.2f?”便于解析

```

if(uwTick-UartTick>900)
{
    UartTick = uwTick;
    sprintf(UART_Data_Buff,
        "A%06.2f?B%06.2f?C%05.2f?D%07.2f?E%09.2f?",
        (Home_1.Volat_Effective),          // A: 电压有效值
        (Home_1.Elect_Effective),          // B: 电流有效值
        (Home_1.Frequency),                // C: 频率
        (Home_1.Power_Total*Home_1.Factor), // D: 有功功率
        (E_T*1.0)/3600000);                // E: 总能耗(kWh)
    HAL_UART_Transmit(&huart3, UART_Data_Buff, sizeof(UART_Data_Buff), 100);
}

void Onenet_Init()
{
    // 1. 发送基础AT指令测试模块
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT\r\n", strlen("AT\r\n"), 100);
    HAL_Delay(1000);

    // 2. 复位ESP8266模块
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT+RST\r\n", strlen("AT+RST\r\n"), 100);
    HAL_Delay(2000); // 延长等待时间确保复位完成

    // 3. 设置Wifi模式为Station模式
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT+CWMODE=1\r\n", strlen("AT+CWMODE=1\r\n"), 100);
    HAL_Delay(1000);

    // 4. 连接到Wifi网络 (SSID和密码已转义)
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT+CWJAP=\"TP-LINK_ED23\", \"xiaobawang1234\" \r\n", strlen("AT+CWJAP=\"TP-LINK_ED23\", \"xiaobawang1234\" \r\n"), 100);
    HAL_Delay(5000); // 延长等待时间确保Wifi连接

    // 5. 配置MQTT客户端 (OneNET认证信息)
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT+MQTTUSERCFG=0,1,\"stm32f103\", \"cF16Dwy2B8\", \"version=2018-10-31&res=pr\", 100);
    HAL_Delay(2000);

    // 6. 连接到OneNET MQTT服务器 (使用安全端口1883)
    HAL_UART_Transmit(&huart2, (uint8_t*)"AT+MQTTCONN=0,\"mqtts.heclouds.com\", 1883, 1\r\n", strlen("AT+MQTTCONN=0,\"mqtts.heclouds.com\", 1883, 1\r\n"), 100);
    HAL_Delay(2000); // 延长等待时间确保MQTT连接

    while (1)
    {
        if(Uart_Flag)
        {
            float voltage, current, frequency, active_power, total_energy;
            sscanf(Re_Data, "A%f?B%f?C%f?D%f?E%f", &voltage, &current, &frequency, &active_power, &total_energy); //数据解析

            sprintf(json_buf,
                "{\"id\":\"%lu\", \"version\":\"1.0\", \"params\":{\""
                "\"voltage\": {\"value\": %.2f}, \""
                "\"current\": {\"value\": %.2f}, \""
                "\"frequency\": {\"value\": %.2f}, \""
                "\"active_power\": {\"value\": %.1f}, \""
                "\"total_energy\": {\"value\": %.2f}}}",
                msg_id++, voltage, current, frequency, active_power, total_energy);

            sprintf(topic, "$sys/%s/%s/thing/property/post", PRODUCT_ID, DEVICE_NAME);

            sprintf(mqtt_cmd, "AT+MQTTPUB=0,\"%s\", \"%s\", 1, 0\r\n", topic, json_buf);
            HAL_UART_Transmit(&huart2, (uint8_t*)mqtt_cmd, strlen(mqtt_cmd), 1000);

            Uart_Flag = 0;
            HAL_UART_Receive_DMA(&huart1, Re_Data, RE_SIZE);
        }
        HAL_Delay(10);
    }
}

```

图 3.2 单片机与 ESP8266 的通信

3.2 云平台上的部署



图 3.3 物模型设置

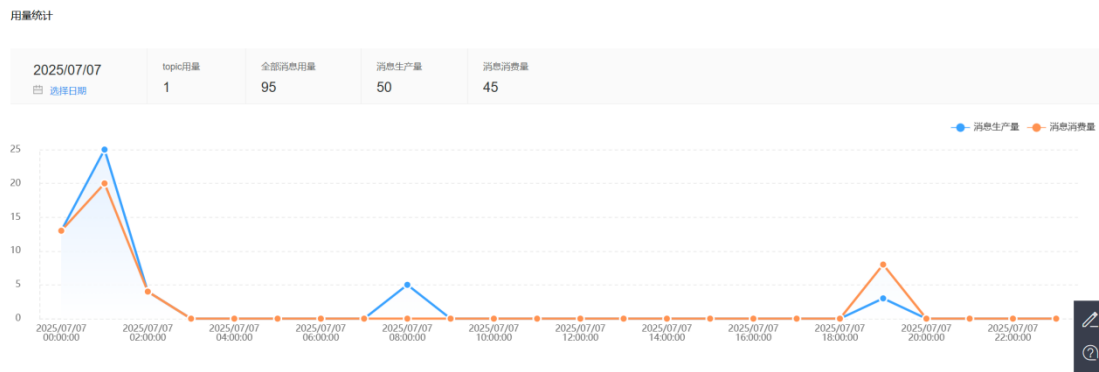


图 3.4 MQ 消息队列

3.3 QT 上位机部署

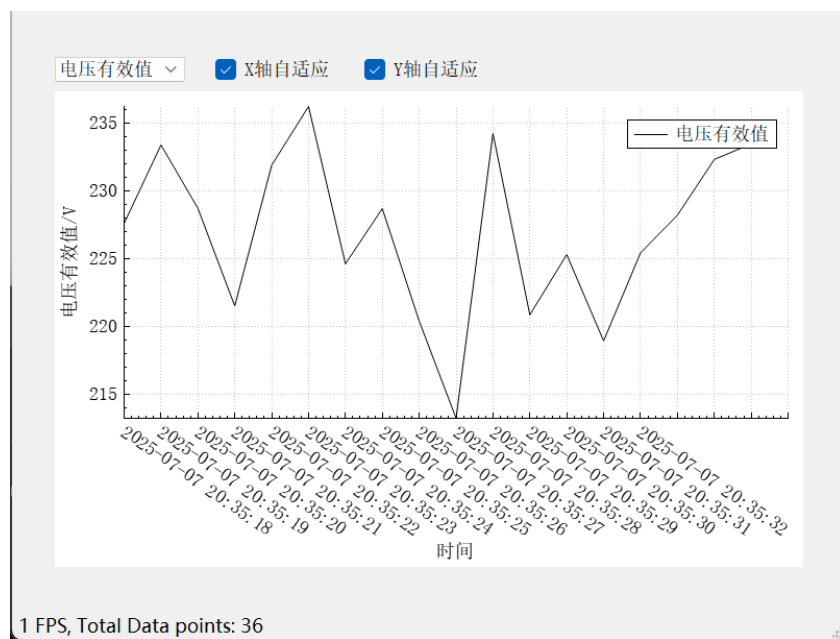


图 3.5 QT 上位机界面

3.4 云平台介绍

3.4.1 OneNET 云平台的优势

能够基于物模型进行开发，规范了数据传输的 json 格式，方便上传与下发。MQ 消息队列中支持 SQL 调试，能够无痛过滤冗余信息。OneNET 还提供 TLS1.2 证书加密，使数据传输更加安全。OneNET 平台提供丰富的文档与技术支持，有助于开发。

3.4.2 OneNET 中数据传输的路径

下位机以 json 格式将数据上传至 OneNET 上的物模型，此前先添加 MQ 消息队列的消费实例，在消费实例中建立 topic，之后在 topic 中添加订阅。在规则引擎中将设备与订阅绑定，即可实现数据的订阅。

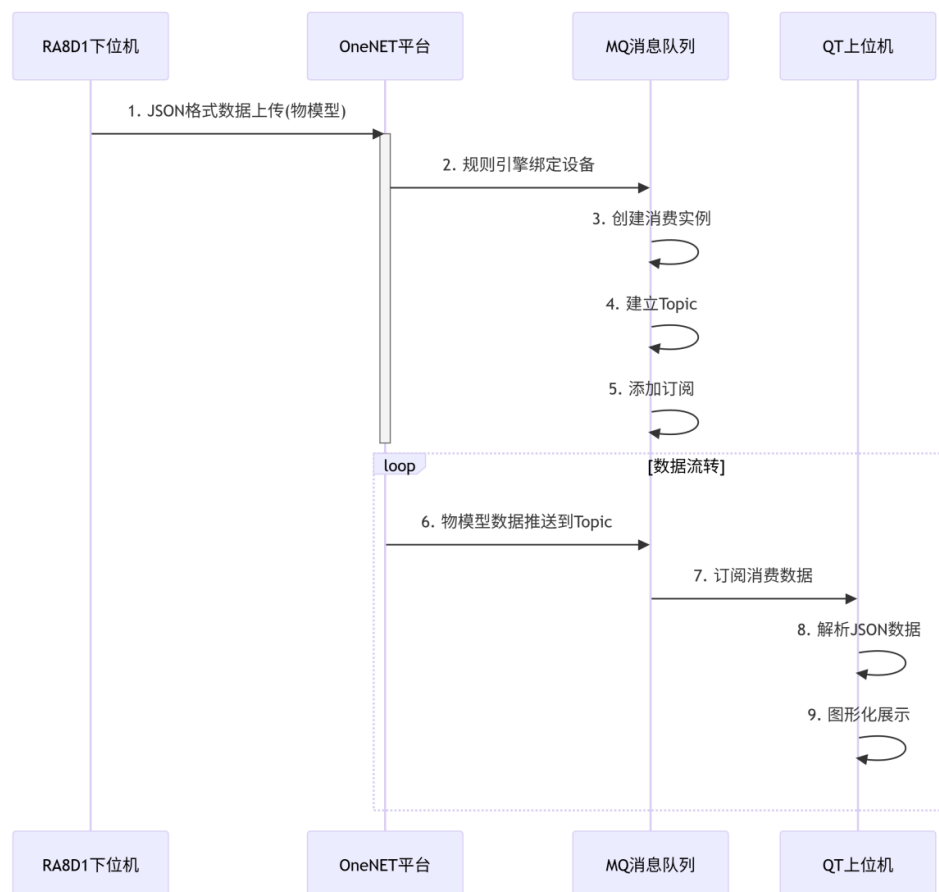


图 3.6 数据传输路径

3.5 QT 上位机介绍

3.5.1 QT 与 OneNET 建立联系

首先创建 QMqttClient 对象作为通信核心，配置 OneNET 服务器地址 183.230.40.96 和加密端口 8883；接着生成平台通行证——用户名设为产品 ID“MQMQMQ”，密码构造为包含版本号、资源路径、过期时间戳、签名方法和 Base64 签名值的认证 Token，客户端 ID 固定为“MQTT”；然后通过 setClientId/setUsername/setPassword 方法绑定这些凭证；最后从指定路径加载 PEM 格式的 SSL 证书“MQ-certificate-release-0711.pem”，建立 TLS 加密通道完成安全连接配置。

```

/* ===== MQTT Broker 配置 ===== */
/*OneNET配置*/
client = new QMqttClient(this); // 考虑将client作为MainWindow的成员变量或指针

/*OneNET地址*/
client->setHostname("183.230.40.96");
client->setPort(8883); //加密端口

/*生成通行证*/
QString username = "MQMQMQ";
QString password = "version=2018-10-31&res=mqs%2FMQMQMQ&et=2666666954&method=md5&sign=dCs3QatWzx%2BHQfPumoSE7A%3D%3D";
QString clientId = "MQTT";

/*佩戴通行证*/
client->setClientId(clientId);
client->setUsername(username);
client->setPassword(password.toUtf8());

/* 设置SSL证书 */
m_certPath = "C:/Users/zyh/Downloads/a57a640bc6e616ea55c54b2db7f5320c (2)/MQ-certificate-release-0711.pem";
loadSSLCertificate(); // 加载证书
  
```

图 3.7 QT 与 OneNET 建立联系

3.5.2 数据清洗

由于从云平台传来的数据可能会出现乱码情况，如图所示：

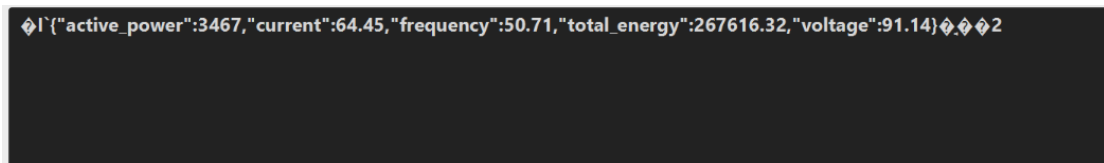


图 3.8 乱码情况演示

故而需要进行数据清洗再进入数据解析阶段。

```
// =====  
// 数据清洗  
// =====  
QByteArray MainWindow::cleanMqttPayload(const QByteArray &payload)  
{  
    // 查找有效的JSON起始位置  
    int startPos = payload.indexOf('{');  
    if (startPos == -1) {  
        qDebug() << "未找到有效JSON起始位置";  
        return QByteArray();  
    }  
  
    // 查找有效的JSON结束位置  
    int endPos = payload.lastIndexOf('}');  
    if (endPos == -1 || endPos <= startPos) {  
        qDebug() << "未找到有效JSON结束位置";  
        return QByteArray();  
    }  
  
    // 截取有效JSON部分  
    QByteArray cleanData = payload.mid(startPos, endPos - startPos + 1);  
  
    // 调试输出清洗前后对比  
    qDebug() << "==== 原始数据 =====";  
    qDebug() << payload;  
    qDebug() << "十六进制:" << payload.toHex();  
    qDebug() << "==== 清洗后数据 =====";  
    qDebug() << cleanData;  
  
    return cleanData;  
}
```

图 3.9 清洗部分代码

3.5.3 数据解析

首先严格校验 JSON 语法和结构完整性，捕获解析错误时输出精准定位信息；其次验证根对象必须包含“params”子对象；接着定义安全提取函数，兼容直接数值与{value:xxx}嵌套格式，对缺失或异常字段返回 NaN；最终将解析成功的电压、电流、功率等电气参数映射存储到结构体数组的对应成员中。

```

/*2.使用JSON翻译器解码*/
//QJsonDocument doc = QJsonDocument::fromJson(payload);
QJsonParseError parseError;
QJsonDocument doc = QJsonDocument::fromJson(payload, &parseError);

//检查json解析是否出错
if(parseError.error != QJsonParseError::NoError)
{
    qDebug() << "JSON解析错误:" << parseError.errorString();
    qDebug() << "错误位置:" << parseError.offset;
    qDebug() << "错误上下文:" << payload.mid(parseError.offset-10, 20);
    return;
}

//格式有误（非json格式）
if (!doc.isObject())
{
    qDebug() << "数据格式有误，无效处理！";
    return;
}

//格式正确，将数据从json格式中提取出来
QJsonObject root = doc.object(); //获取根对象

/*3.解析从OneNET中获取到的数据*/
/* 检查并获取params对象 */
if (!root.contains("params") || !root["params"].isObject()) {
    qDebug() << "错误：未找到有效的params对象";
    return; // 或处理错误
}
QJsonObject params = root["params"].toObject();

auto safeGetDouble = [&params](const QString &key) {
    if (!params.contains(key)) {
        qDebug() << "字段" << key << "不存在";
        return std::numeric_limits<double>::quiet_NaN();
    }

    QJsonValue value = params.value(key);
    if (value.isDouble()) {
        return value.toDouble();
    } else if (value.isObject()) {
        QJsonObject obj = value.toObject();
        if (obj.contains("value")) {
            return obj["value"].toDouble(0.0);
        }
    }
    qDebug() << "字段" << key << "格式不支持";
    return std::numeric_limits<double>::quiet_NaN();
};

Home_1[add_data_num].Vol_Effective = safeGetDouble("voltage");
Home_1[add_data_num].Electricity_Effective = safeGetDouble("current");
Home_1[add_data_num].Power = safeGetDouble("active_power");
Home_1[add_data_num].Energy_Total = safeGetDouble("total_energy");
Home_1[add_data_num].Frequency = safeGetDouble("frequency");

```

图 3.10 数据解析部分代码

4 完成情况及性能参数

4.1 整体介绍

系统以 RA8D1 为主控核心，通过双通道 ADC 同步采集电压 / 电流信号，结合 DMA 双缓冲区“乒乓操作”保障数据完整性。依托 RA8D1 的 DSP 与 FPU 单元，调用 1024 点 FFT 算法完成频谱分析，提取 1-12 次谐波分量，计算有效值、有功功率、功率因数及 THD 等核心电力参数。通信层面，通过 UART 连接 ESP8266 模块实现 2.4G 无线传输，数据封装为 JSON 格式上传至 QT 上位机；本地支持 LCD 屏实时显示参数。硬件集成信号调理电路（互感器隔离、滤波）与继电器控制模块，可实现过载保护。

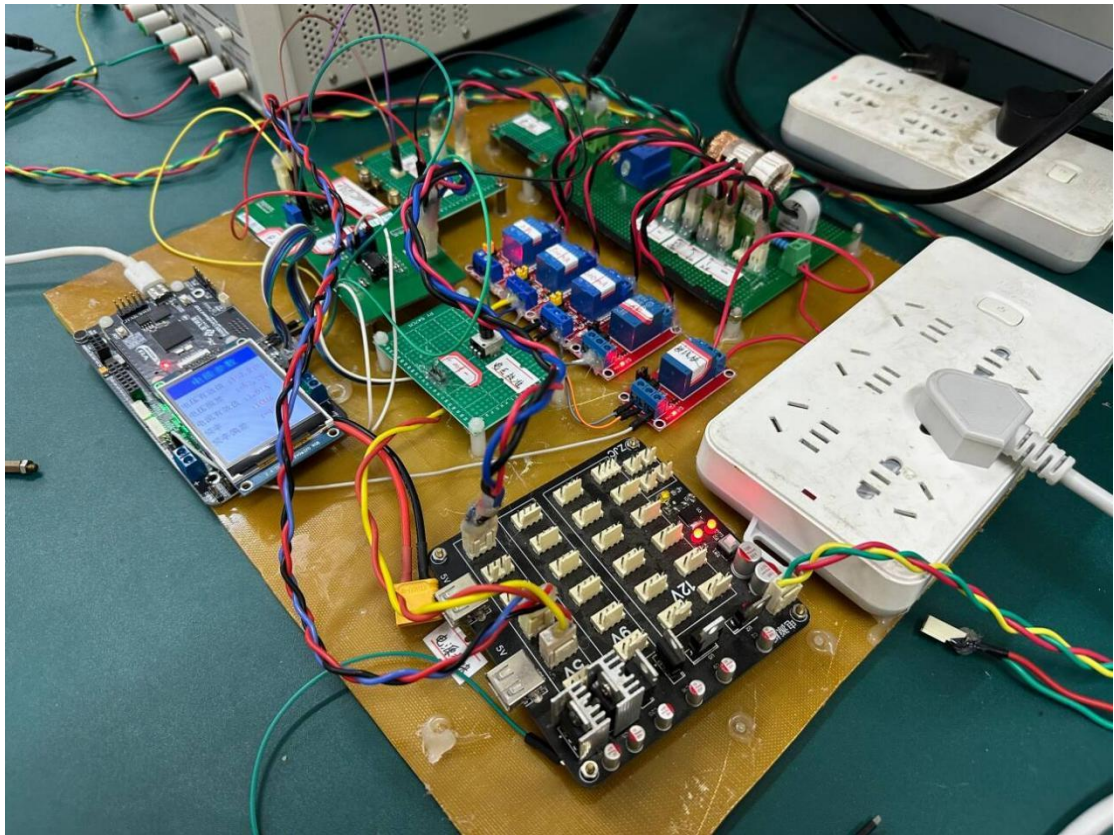


图 4.1 系统斜 45° 全局性照片

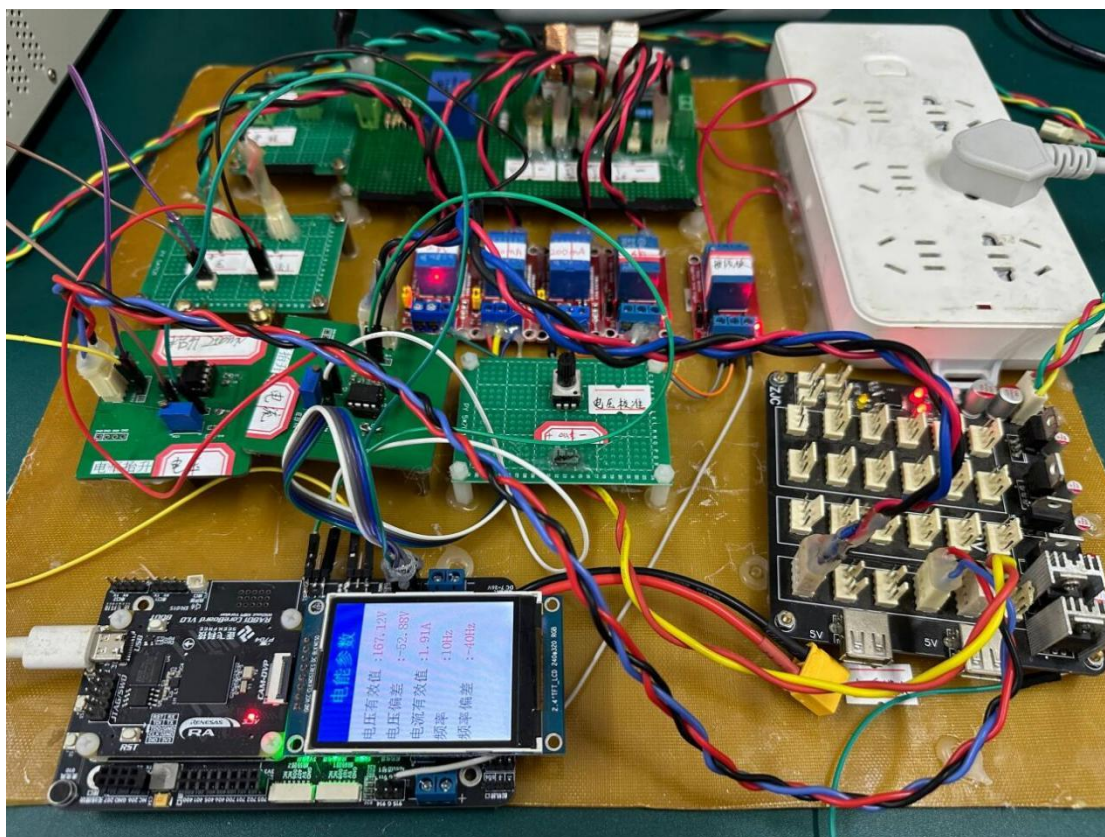


图 4.2 系统正面全局性照片

4.2 工程成果（分硬件实物、软件界面等设计结果）

4.2.1 电路成果

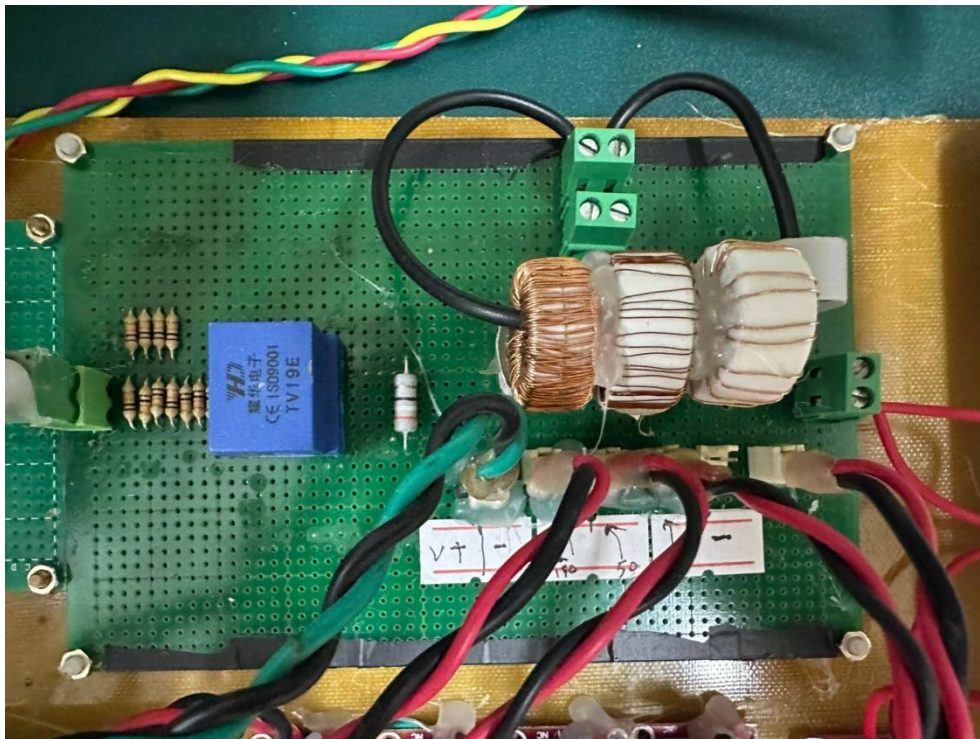


图 4.3 变压器与电流电压互感电路

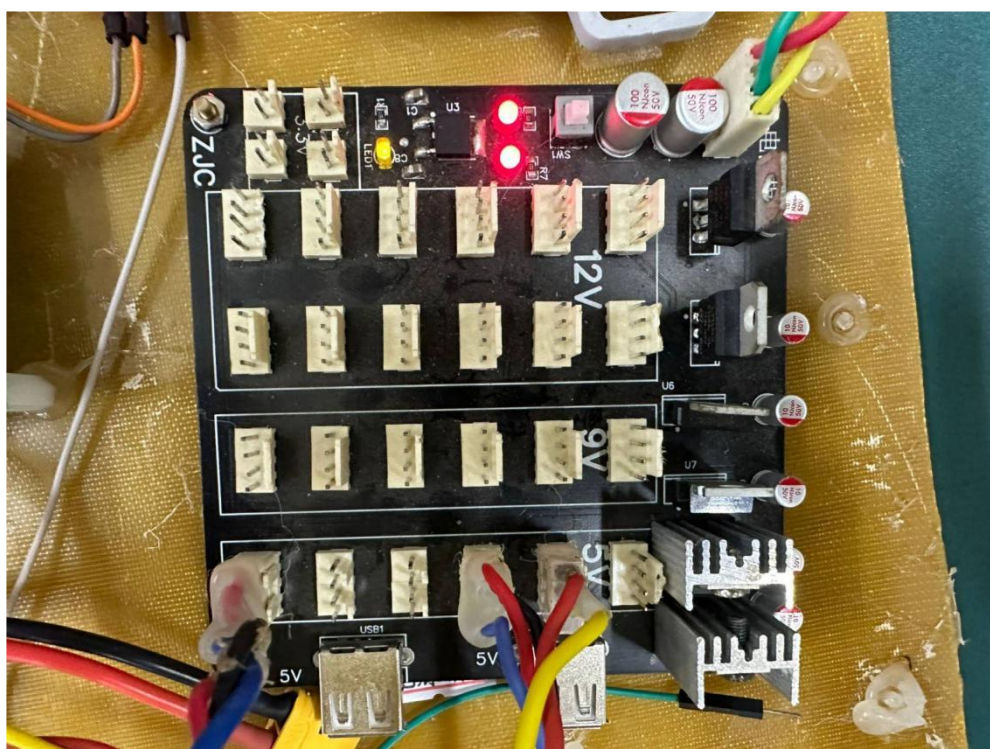


图 4.4 15V 转 5V 供电电路

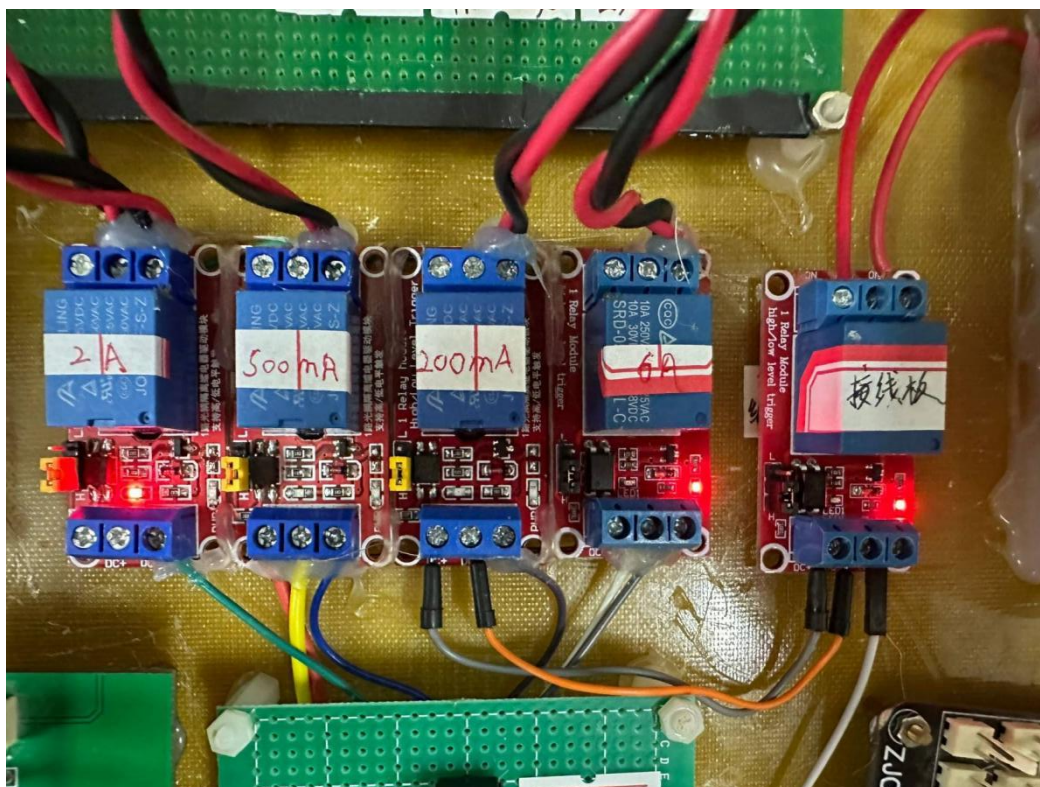


图 4.5 继电器电流选择与功率保护电路

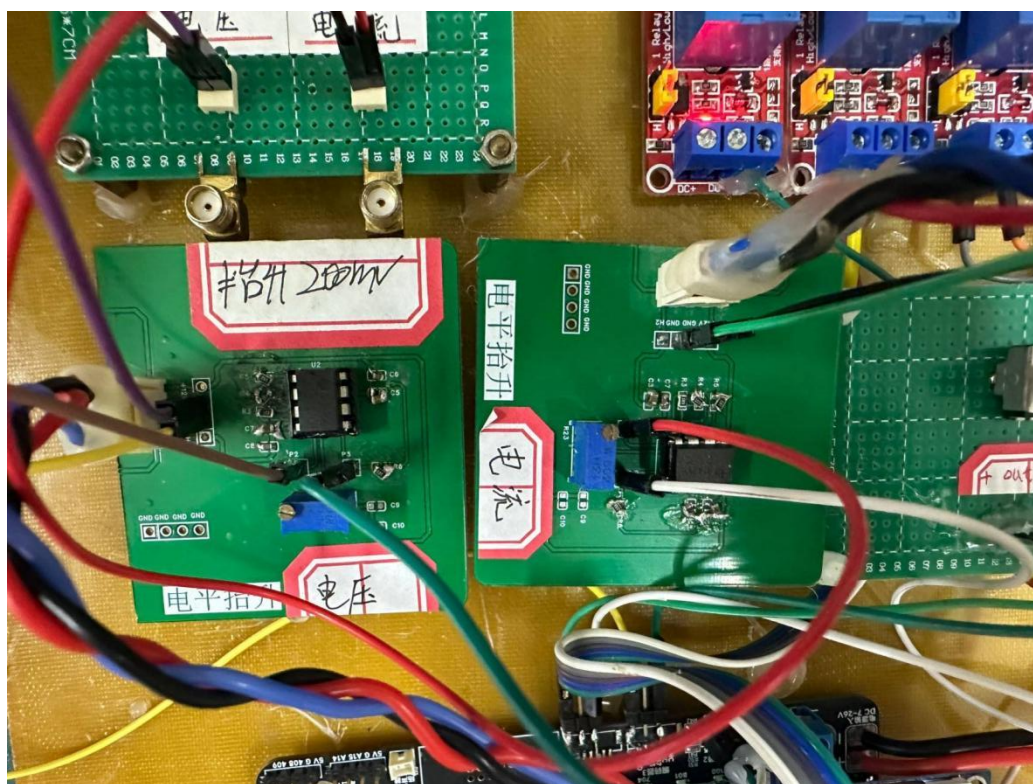


图 4.6 电压抬升电路

4.2.2 软件成果



图 4.7 电能参数界面

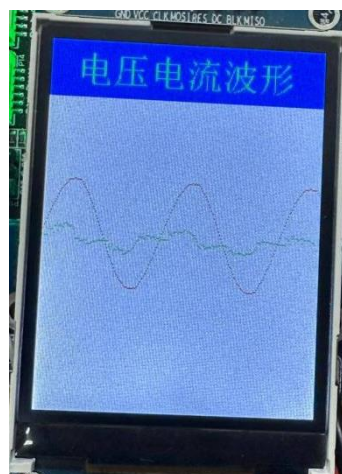


图 4.8 电压电流波形界面



图 4.9 功率参数界面



图 4.10 谐波分析界面

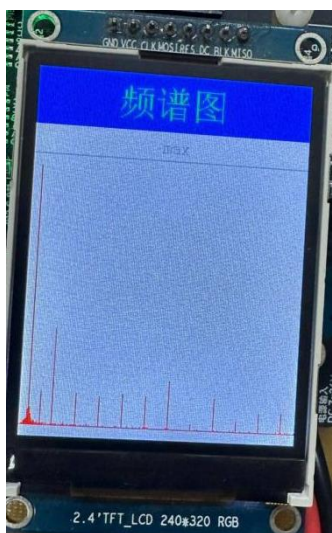


图 4.11 频谱图界面

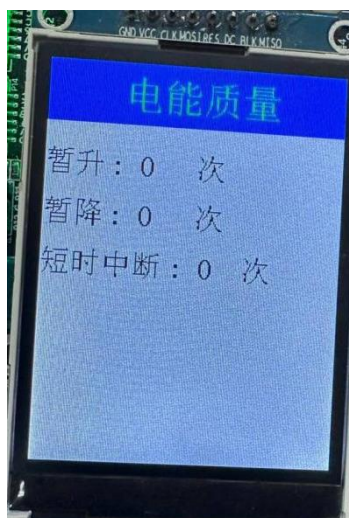


图 4.12 电能质量界面



图 4.13 耗能界面

4.3 特性成果



图 4.14 电流采样值与标准值对比



图 4.15 接入 1500w 电热水壶后功耗



图 4.16 接入 1500w 电热水壶后电流

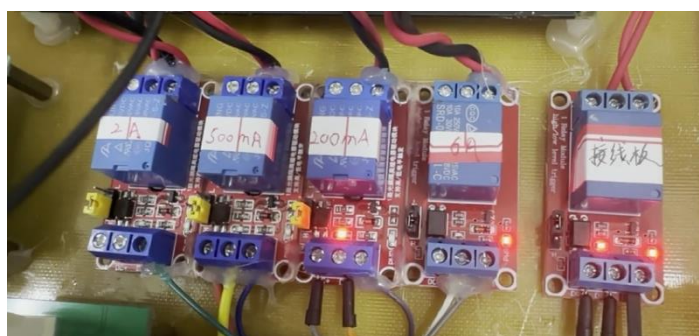


图 4.17 自动档位选择系统正常工作



图 4.18 功率达到 1000w 以后自动断电

5 总结

5.1 可扩展之处

(1) 硬件扩展：可利用 RA8D1_AI_KIT 开发板的接口资源进一步拓展监测功能。通过 I2C 接口接入温湿度传感器、烟雾探测器，将用电安全监测与环境监测结合；外接高精度 ADC 模块，进一步提升电参数测量精度。此外，借助 DVP 接口连接摄像头，实现用电现场可视化监控，为远程管理提供更多信息。

(2) 软件功能升级：在软件层面，可优化 FFT 算法，引入更高级的频谱分析技术，如小波变换，提高对复杂信号的分析能力；拓展谐波分析范围，从 1 - 15 次提升至更高次，以满足更严格的电力质量监测需求。增加机器学习算法，基于历史用电数据训练模型，实现用电行为预测，提前预警异常用电。同时，完善网页端功能，增加数据导出、生成报表等功能，方便用户对数据进行深度分析。

(3) 通信协议与网络融合：可增加对 5G、NB - IoT 等通信协议的支持，适应不同场景下的数据传输需求；探索与区块链技术结合，实现数据的安全存储与可信共享；支持与其他物联网平台对接，实现多系统间的数据交互与协同管理，提升系统兼容性和通用性。

5.2 心得体会

在基于 RA8D1_AI_KIT 开发板实现 FFT 算法单项电能分析仪系统的研发过程中，我深刻体会到硬件与软件深度协同的重要性。开发初期，信号采集的准确性是最大挑战。通过反复调试电流 / 电压互感器与电平抬升电路参数，才将 0-250V 电压、0-4A 电流信号精准调理至 ADC 适配范围，配合 RC 滤波电路，成功将工频干扰抑制在可接受范围内。这一过程让我意识到，硬件设计不仅要理论正确，更需结合实际环境反复验证。

在算法实现环节，利用 RA8D1 芯片的 DSP 与 FPU 特性优化 FFT 算法时，遇到了性能瓶颈。起初直接移植的算法运行效率较低，无法满足实时性要求。通过查阅官方文档、参考开源优化方案，对数据缓存策略、运算流程进行重构，最终将 1024 点 FFT 处理时间压缩至 5ms 以内，谐波分析精度达到设计标准。这一突破让我认识到，硬件性能的充分发挥需要针对性的软件优化。

通信模块开发同样充满挑战，ESP8266 与 QT 上位机的融合使用，涉及网络配置、数据封装与云端对接等多个环节。在调试断线重连功能时，曾因网络延迟导致数据重复上传，通过增加时间戳校验与状态机管理，才实现稳定可靠的数据传输。Node-RED 网页可视化设计则让我体会到低代码开发的便捷，通过模块化搭建，快速实现了数据动态展示与历史查询功能。

整个项目中，RA8D1_AI_KIT 开发板的集成化设计大幅缩短了开发周期。核心板与扩展板的丰富接口，使传感器连接、显示模块驱动等工作变得简单高效。此次研发不仅让我掌握了电力参数监测系统的完整开发流程，更深刻理解了理论与实践结合的重要性。从硬件选型到算法优化，从通信协议到可视化设计，每个环节都需要严谨的态度与创新思维。未来，我将继续探索系统的优化空间，如进一步降低功耗、拓展功能模块，让该系统在更多领域发挥价值。

参考文献

- [1] 童诗白, 华成英主编. 《模拟电子技术基础》 高等教育出版社, 2006.
- [2] 塞尔吉欧·弗朗哥(美)著 《基于运算放大器和模拟集成电路的电路设计》 西安交通大学出版社
- [3] 稻叶保(日)著 《模拟技术应用技巧 101 例》 科学出版社
- [4] 卢旭坤.关于高精度 ADC 关键技术的探讨[J].通讯世界,2024,31(12):25-27.
- [5] 范若涵.物联网技术在电力设备实时监控系统设计中的应用[J].现代传输,2025,(02):65-69.